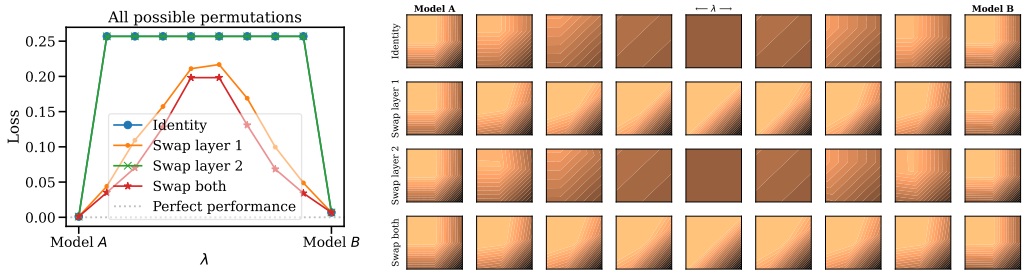

Figure 6: **A counterexample to universal LMC.** There exist models such that no possible permutation of weights allows for linear mode connectivity. *Left:* performance of all possible linear interpolations between the two models. *Right:* A visualization of the prediction functions $f(\boldsymbol{x})$ through each linear sweep. Each row corresponds to one of the four possible permutations, and each column corresponds to a value of λ, the linear interpolant. The existence of such cases suggests that linear mode connectivity is an artifact of SGD.

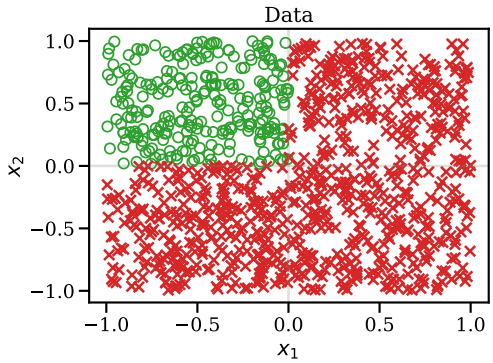

Figure 7: The counterexample classification problem data.

1. MLP (3 layers, 512 units each): 3 seconds
2. ResNet50 (1× width): 33 seconds
3. ResNet20 (32× width): 194 seconds

In addition, we tested the ability of Algorithm 1 to recover a known, randomly selected permutation. In a handful of experiments we found that Algorithm 1 was able to exactly recover the known, random permutation in just 3-4 of passes over the layers.

A.6 Counterexample Details

Consider a simple 2-dimensional classification task. Our data points are drawn $\boldsymbol{x} \sim \text{Uniform}([-1,1]^2)$ and $y = \mathbf{1}_{x_1<0 \text{ and } x_2>0}$. Figure 7 provides a visualization of a sample of such data.

We utilize an MLP architecture consisting of two hidden layers, with two units each, and ReLU nonlinearities. Consider two weight assignments that both achieve a perfect fit to the data:

$$f_A(\boldsymbol{x}) = \begin{bmatrix}-1 & -1\end{bmatrix} \sigma\left(\begin{bmatrix}-1 & 0\\ 0 & 1\end{bmatrix} \sigma\left(\begin{bmatrix}-1 & 0\\ 0 & -1\end{bmatrix} \boldsymbol{x} + \begin{bmatrix}1\\ 0\end{bmatrix}\right) + \begin{bmatrix}1\\ 0\end{bmatrix}\right) \tag{4}$$

$$f_B(\boldsymbol{x}) = \begin{bmatrix}-1 & -1\end{bmatrix} \sigma\left(\begin{bmatrix}1 & 0\\ 0 & -1\end{bmatrix} \sigma\left(\begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix} \boldsymbol{x} + \begin{bmatrix}0\\ 1\end{bmatrix}\right) + \begin{bmatrix}0\\ 1\end{bmatrix}\right). \tag{5}$$

We predict a positive label when $f(\boldsymbol{x}) \geq 0$ and a negative label otherwise.

Intuitively, these networks are organized such that each layer makes a classification whether $\boldsymbol{x}_1 < 0$ or $\boldsymbol{x}_2 > 0$. In model A, the first layer tests whether $\boldsymbol{x}_2 > 0$, and the second layer tests whether $\boldsymbol{x}_1 < 0$, whereas in model B the order is reversed. With a bit of algebra, it is possible to see that both f_A and f_B achieve perfect performance. However, no possible permutation of units results in linear mode connectivity between f_A and f_B. We visualize all possible permutations in Figure 6.

We claim that this example, and the underlying trick, are simple enough to be embedded into larger models. For example, this could trivially be extended to ResNets where different subsets of layers could be set to identity functions.

As discussed in Section 4, the existence of adversarial basins in the loss landscape has interesting consequences for our understanding of loss landscape geometry. In particular, we argue that this implies that common optimization algorithms are conveniently biased towards solutions that admit LMC. However, the precise connection between optimization algorithms and linear mode connectivity remains unclear.

This counterexample does not constitute a contradiction of the conjecture in Entezari et al. (2021). To be more precise, Conjecture 1 of Entezari et al. (2021) proposes that there exists some subset, $\mathcal{S}$, of parameter space such that every pair of elements in $\mathcal{S}$ can be linearly mode connected (after some permutation of units), and that with high probability SGD solutions are contained in $\mathcal{S}$. The example presented in this section does not contradict Entezari et al. (2021)'s conjecture, but instead illustrates that the restriction to SGD solutions is a "load-bearing" element of the conjecture.

A.7 On the Failures of Greedy Uni-directional Matching

In contrast to prior work (Pittorino et al., 2022; Singh & Jaggi, 2020; Wang et al., 2020), we eschew greedy uni-directional, single-pass matching between models. Instead we derive a weight matching algorithm from a principled, yet computationally infeasible optimization problem. In contrast to prior work, our method can be viewed as "bi-directional": it selects unit associations based on weights in all relevant layers, not just in the immediately previous layer. In this section, we describe benefits of our holistic approach, including an example problem showing the failure modes of greedy uni-directional matching. We find that matching across all layers simultaneously allows our weight matching algorithm (Algorithm 1) to exploit units' relationships with downstream weights in a way that greedy uni-directional matching cannot.

Concretely, greedy uni-directional matching begins at the first layer and computes a matching, $\boldsymbol{P}_1$, considering only $\boldsymbol{W}_1^{(A)}, \boldsymbol{W}_1^{(B)}$. After matching, $\boldsymbol{P}_1$ is applied throughout the remainder of the network. Then, we proceed through the layers in order repeating this process.

We compare our Algorithm 1 to greedy uni-directional matching experimentally in Appendix A.7.1 and present a theoretical counterexample that illustrates the advantages of our method in Appendix A.7.2.

A.7.1 Experimental Comparison

In order to further evaluate our performance relative to prior work, and Pittorino et al. (2022); Singh & Jaggi (2020) in particular, we explored experimental comparisons with VGG11 models trained on CIFAR-10 and on ResNet50 models trained on ImageNet.

VGG11 models trained on CIFAR-10. Following an exact reproduction of experiment from Table 1 of Singh & Jaggi (2020), we merged the model weights released along with their paper. We present results in Table 2. We found that our weight matching method outperforms the "wts" method of Singh & Jaggi (2020) in both implementation speed and model performance when reproducing their experiment with their trained model weights.

ResNet50 models trained on ImageNet. We applied OT-Fusion ("wts") to the ImageNet experiment that we consider in Section 5.1. We present results in Figure 8. We found the OT-Fusion method resulted in models with 1.38% top-1 accuracy on ImageNet, only marginally improving over naïve averaging. On the other hand, we achieve 51.01%.

BatchNorm statistics were recalculated after interpolation for all methods shown.

Method	Test accuracy (↑)	Run-time (↓)
OT-Fusion (Singh & Jaggi, 2020)	85.98%	2.86s
Weight matching (ours)	**86.57%**	**0.64s**

Table 2: **VGG11/CIFAR-10 performance relative to Singh & Jaggi (2020).** We found that our weight matching method outperforms the "wts" method of Singh & Jaggi (2020) in both implementation speed and model performance when reproducing one of their experiments with their published model weights. Our implementation is 4.5× faster, and produces a solution with better model performance.

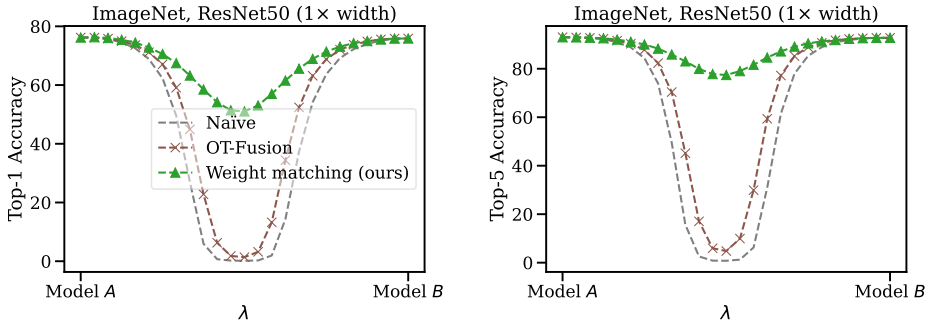

Figure 8: **ResNet50/ImageNet performance relative to Singh & Jaggi (2020).** We found the OT-Fusion method resulted in models with 1.38% top-1 accuracy on ImageNet, only marginally improving over naïve averaging. On the other hand, we achieve 51.01%.

Although a number of factors may be responsible for the difference in performance between weight matching and OT-Fusion, we found the number of alignment passes made over the network layers to have a substantial impact. OT-Fusion is inherently limited to a single pass over the layers. On the other hand, we are not limited to any specific number of optimization passes and instead continue until convergence (convergence is guaranteed by Lemma 2). For comparison, if our weight matching algorithm is artificially handicapped to a single pass over the layers, we achieve a similarly low $\sim 7\%$ top-1 accuracy.

A.7.2 An Example Failure Case

Consider two networks, A and B, with the objective that they capture the identity function $f(x) = x$,

$$f_{\Theta_A}(x) = \begin{bmatrix}1 & 0\end{bmatrix} \begin{bmatrix}1 & 0 \\ 0 & \epsilon\end{bmatrix} \begin{bmatrix}1 \\ 1+\epsilon\end{bmatrix} x \tag{6}$$

$$f_{\Theta_B}(x) = \begin{bmatrix}0 & 1\end{bmatrix} \begin{bmatrix}0 & 0 \\ 0 & 1\end{bmatrix} \begin{bmatrix}1 \\ 1+\epsilon\end{bmatrix} x \tag{7}$$

where $\epsilon > 0$ is some negligible constant. It can be seen that these reduce to $f_{\Theta_A}(x) = x$ and $f_{\Theta_B}(x) = (1+\epsilon)x$.

When aligning these models there are two possible opportunities for permutation, $\pi = \{\boldsymbol{P}_1, \boldsymbol{P}_2\}$. The permuted model B then has the form

$$f_{\pi(\Theta_B)}(x) = \left(\begin{bmatrix}0 & 1\end{bmatrix} \boldsymbol{P}_2^\top\right) \left(\boldsymbol{P}_2 \begin{bmatrix}0 & 0 \\ 0 & 1\end{bmatrix} \boldsymbol{P}_1^\top\right) \left(\boldsymbol{P}_1 \begin{bmatrix}1 \\ 1+\epsilon\end{bmatrix}\right) x \tag{8}$$

Now, aligning with greedy uni-directional matching will result in the alignment $\pi_{gud} = \{\boldsymbol{P}_1 = \boldsymbol{I}, \boldsymbol{P}_2 = \boldsymbol{I}\}$. On the other hand, our weight matching method (Algorithm 1) results in $\pi_{wm} = \left\{\boldsymbol{P}_1 = \begin{bmatrix}0 & 1 \\ 1 & 0\end{bmatrix}, \boldsymbol{P}_2 = \begin{bmatrix}0 & 1 \\ 1 & 0\end{bmatrix}\right\}$, regardless of the algorithm's execution order.